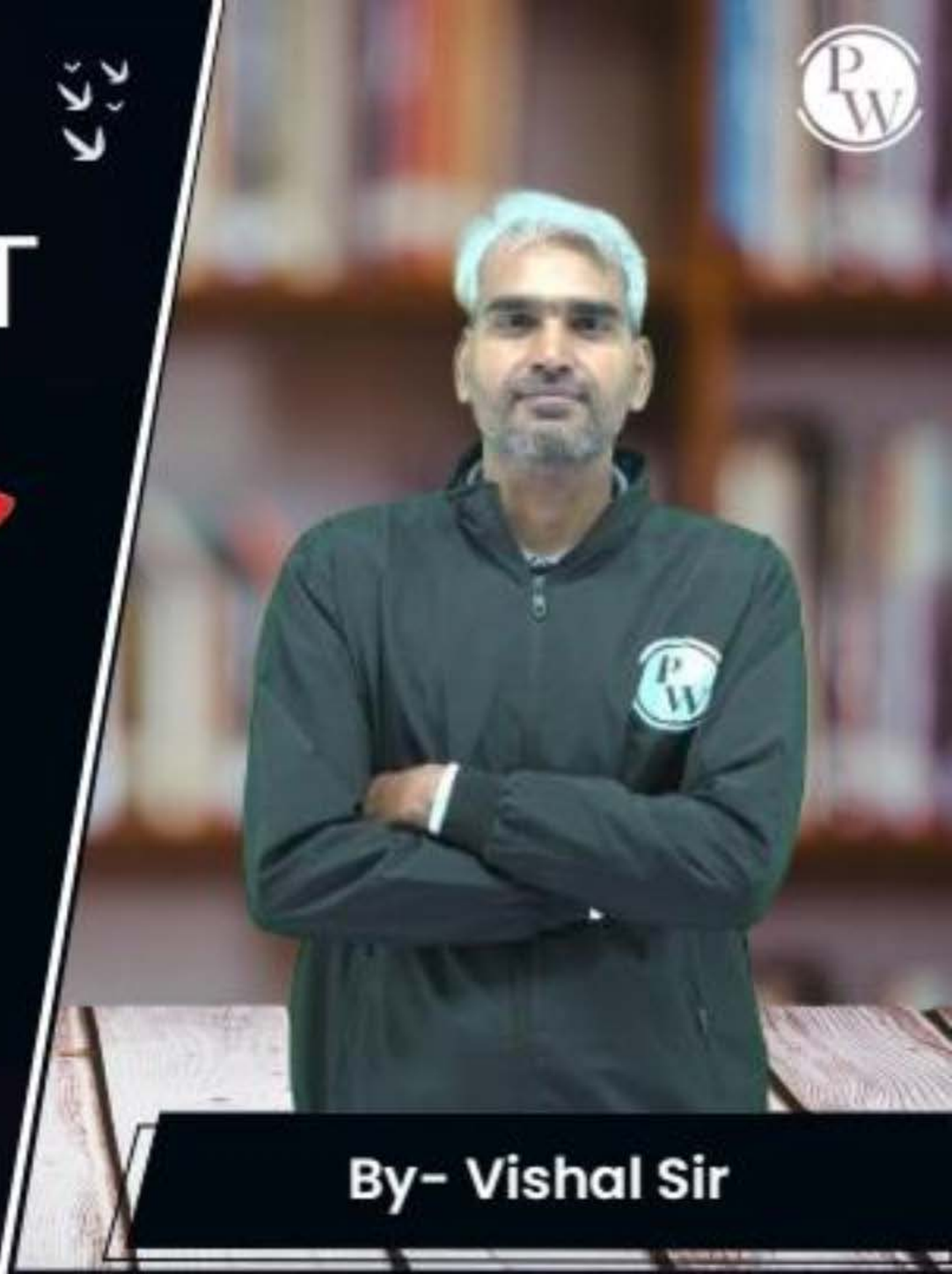
Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 07



By- Vishal Sir

Recap of Previous Lecture



Topic Types of Top-down Parsers

Topic Introduction to LL(1) Parser

Slide

Topics to be Covered



- Topic: LL(1) Parsing Table construction
- Topic: LL(1) Parsing Algorithm



Topic : Top-down Parser

- In Top Down parsing, we will try to derive the given string starting from the start symbol.
- If any parser uses top down approach for parsing the string, then it follows leftmost derivation in the construction of parse tree.
- Top Down Parser can see only one symbol of the input string at a time from left to right.



Topic : Type of Top-down Parsers

Top down Parser

- With backtracking (Brute Force technique)
  - Recursive Descent Parser With back tracking
- Without back tracking (Predictive Parsing)
  - Recursive Descent Parser With no back tracking
  - Non-Recursive Predictive Parser or LL(1) Parser

Table driven Parser

Slide



Topic : LL(1) Parser

It is a Top-down parser (without backtracking) {using Table}

aka. Non-Recursive Predictive Parser

L L (1) Parser

- 1 look-ahead Symbol.
- Leftmost derivation
- Left to Right scan

Slide



Topic : L L (1) Parser

A grammar can be parsed using LL(1) parser only if,

I) Grammar is unambiguous grammar.

II) Grammar is free from left recursion.

III) Grammar does not contain common prefix problem.

Necessary Conditions but not sufficient

There are some unambiguous grammars which can not be parsed using top-down parser

Slide



Topic : Construction of LL(1) Parsing Table



Let M is the LL(1) Parsing table

For each production $A \to \alpha$ repeat the following,

1. Add $A \to \alpha$, under $M[A, b]$, $\forall b \in first(\alpha)$.
2. If $\epsilon \in first(\alpha)$, then add $A \to \epsilon$ under $M[A, c]$, $\forall c \in Follow(A)$

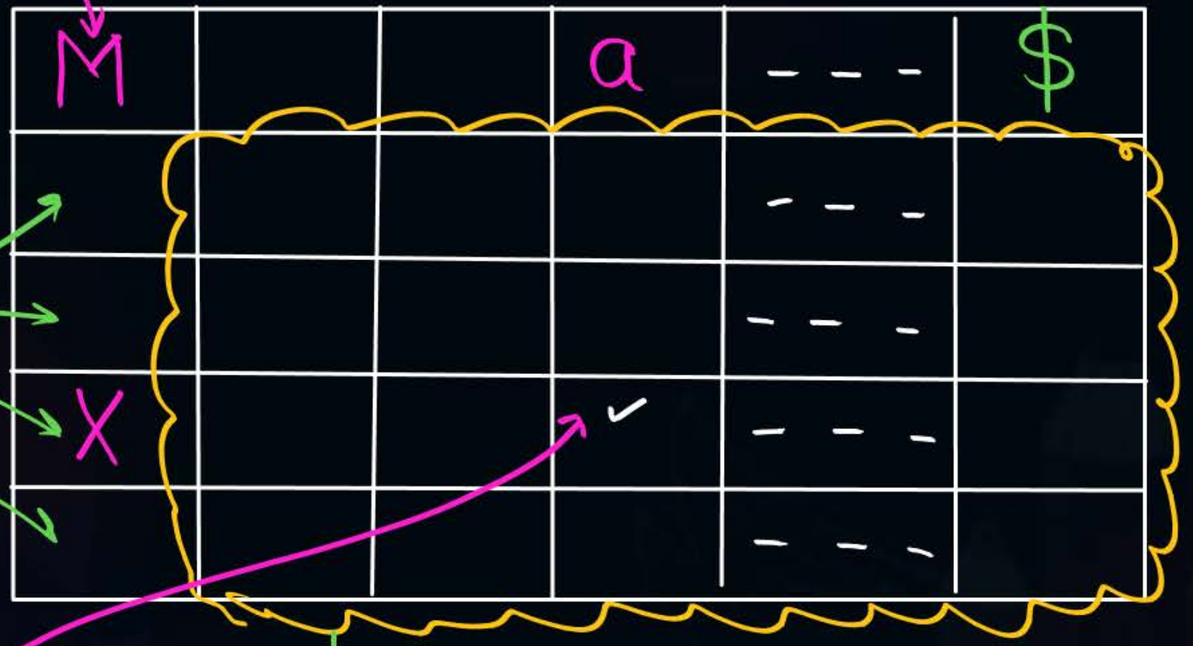
Slide



Topic : Construction of LL(1) Parsing Table

Name of table

Non-null terminals of the gramma



Non-terminals of the gramma

M[X, a]

entry w.r.t. non-terminal X and terminal 'a'

Size of LL(1) Parsing table = (No. of non terminals) * (No. of non-epsilon terminals + 1)

because of $

Slide



Topic : Construction of LL(1) Parsing Table

Construct the LL(1) parsing table for following grammar

$S \rightarrow (L) \Rightarrow first((L)) = \{(\} \Rightarrow \therefore M[S, (] = S \rightarrow (L)$

$S \rightarrow a \Rightarrow first(a) = \{a\} \Rightarrow \therefore M[S, a] = S \rightarrow a$

$S \rightarrow (L) / a \quad first(S) = \{(, a\}$

$L \rightarrow SL' \Rightarrow first(SL') = \{(, a\}$

$L' \rightarrow , SL' / \epsilon$

M	a	(	)	,	$
S	$S \rightarrow a$	$S \rightarrow (L)$			
L	$L \rightarrow SL'$	$L \rightarrow SL'$			
L'			$L' \rightarrow \epsilon$	$L' \rightarrow , SL'$	

No. of non-terminals = 3 $\{S, L, L'\}$

No. of non epsilon terminals = 4 $\{a, (,), ,\}$

Size of LL(1) table $= 3 * (4+1) = 15$

$L' \rightarrow , SL' \Rightarrow first(,SL') = \{,\}$

$L' \rightarrow \epsilon \Rightarrow \therefore follow(L') = \{)\}$

Slide

* All blank entries in LL(1) parsing table will correspond to parsing error (Syntax Error) during parsing



Topic : Construction of LL(1) Parsing Table

Construct the LL(1) parsing table for following grammar

$E \rightarrow TE'$

$E' \rightarrow +TE' / \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \varepsilon$

$F \rightarrow id / (E)$

M	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Slide

$A \rightarrow Aa \Rightarrow \text{first}(Aa) = \{b\}$

$A \rightarrow b \Rightarrow \text{first}(b) = \{b\}$

$A \rightarrow Aa / b$

Left Recursion

Will always lead to more than one production in at least one entry of LL(1) Parsing table

M	b	----
A	$A \rightarrow Aa$ $A \rightarrow b$	

$A \rightarrow aA / ab$

Common prefix

$A \rightarrow aA \Rightarrow first(aA) = \{a\}$

$A \rightarrow ab \Rightarrow first(ab) = \{a\}$

	- - - -	a
A		$A \rightarrow aA$ $A \rightarrow ab$



Topic : LL(1) Grammar

If all the strings generated by the grammar can be parsed using LL(1) parser, then grammar is called LL(1) grammar

1. If each entry of LL(1) parsing table contains at most one production, then given grammar is LL(1) grammar.
   i.e. blank (or) one production
2. If any entry of LL(1) parsing table contains more than one production, then given grammar is not LL(1) grammar.
3. All LL(1) grammars are unambiguous, but all unambiguous grammars need not be LL(1).
4. If grammar contains left recursion or common prefix, then we will have multiple production in at least one entry of LL(1) parsing table. Therefore, left recursive grammars or grammars with common prefix can not be LL(1) grammars.

Slide



Topic : LL(1) Parsing Algorithm

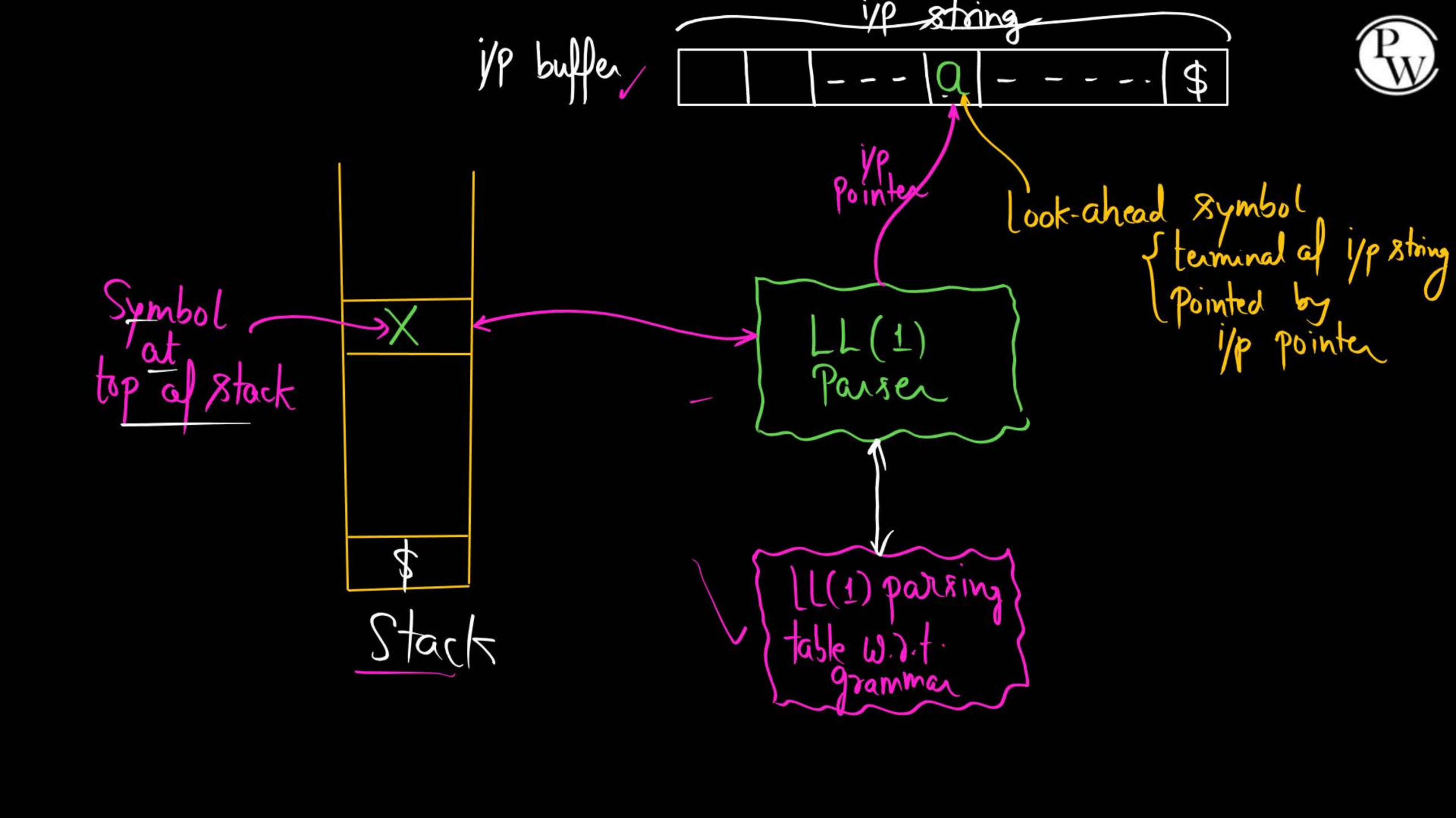
LL(1) parsing will use
① One stack ③ LL(1) Table
② i/p buffer ④ LL(1) parsing Algo

Initially PUSH $ into the stack and then PUSH starting symbol of the grammar into the stack.

Let X is the top of the stack symbol and 'a' be the look-ahead symbol pointed by input pointer, and let M is the LL(1) parsing table, then:

1. If X = a = $ then parsing is is successful
2. If (X = a) ≠ $ then POP X from stack and increment input pointer. (terminals matched)
3. If X is a non-terminal and M[X, a] contain the production X→uvw then replace X by uvw in the reverse order {i.e., u should appear at the top of stack}.
4. If X is a non-terminal and M[X, a] is a blank entry in the table, then indicate syntax error.
5. If X is a terminal and X ≠ a, then indicate syntax error.

Slide


i/p string
i/p buffer
a
$
i/p Pointer
Look-ahead Symbol
{ terminal of i/p string Pointed by i/p pointer
Symbol at top of Stack
X
$
Stack
LL(1) Parser
LL(1) parsing table w.r.t. grammar

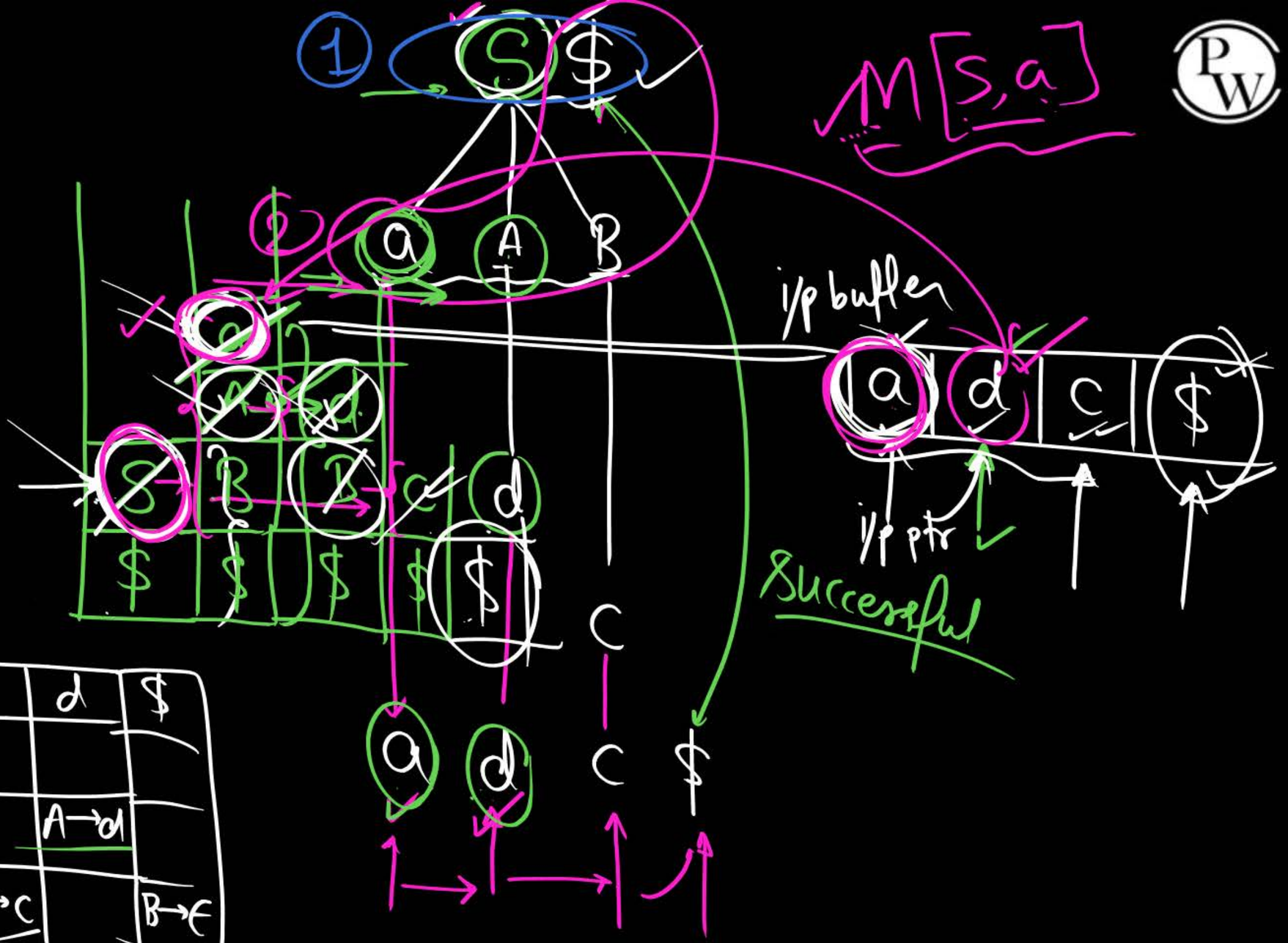
$S \rightarrow aAB$

$A \rightarrow b/d$

$B \rightarrow c/\epsilon$

i/p a d c $

	a	b	c	d	$
S	S→aAB				
A		A→b		A→d	
B			B→c		B→ϵ





Topic : LL(1) Parsing Algorithm

Consider the following grammar

$E \rightarrow TE'$

$E' \rightarrow +TE'/\varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT'/\varepsilon$

$F \rightarrow id/(E)$

M	id	+	*	(	)	$
E	E→TE'					
E'						
T						
T'						
F						

Try parsing the input string $id + id * id\$$ using LL(1) parser.

Slide



2 mins Summary

Topic LL(1) Parser table construction

Topic LL(1) Parsing Algorithm


THANK - YOU
Slide